VS Code Claude Code 시작 가이드

VS Code에서 Claude Code 확장 프로그램을 설치하고, 이 프로젝트 템플릿을 활용해 AI 기반 개발 워크플로우를 시작하는 방법을 안내합니다.

목차

1. 사전 준비
2. Claude Code 확장 프로그램 설치
3. Claude Code 첫 실행 및 로그인
4. 템플릿 구조 이해하기
5. 프로젝트 초기 설정 (SETUP.md)
6. 핵심 워크플로우
7. 에이전트 시스템
8. 스킬 (슬래시 명령어)
9. 메모리 시스템
10. 실전 예시: 첫 번째 페이즈 실행

---

1. 사전 준비

시작하기 전에 다음 항목이 필요합니다:

- **VS Code** 최신 버전 (1.90 이상 권장)
- **Anthropic 계정** — console.anthropic.com에서 가입
  - 또는 Claude Pro/Max 구독 (claude.ai)
- **Node.js** 18 이상 (Claude Code가 내부적으로 Node.js를 사용)
- **Git** 설치 및 기본 설정 완료

---

2. Claude Code 확장 프로그램 설치

방법 A: VS Code 마켓플레이스에서 설치

1. VS Code를 엽니다.
2. 왼쪽 사이드바에서 **확장(Extensions)** 아이콘을 클릭합니다 (단축키: `Ctrl+Shift+X`).
3. 검색창에 **"Claude Code"**를 입력합니다.
4. **Anthropic**이 게시한 "Claude Code" 확장 프로그램을 찾습니다.
5. **Install** 버튼을 클릭합니다.
6. 설치 완료 후 VS Code를 재시작합니다.

방법 B: 명령줄에서 설치

터미널에서 다음 명령어를 실행합니다:

```
code --install-extension anthropic.claude-code
```

설치 확인

설치가 완료되면:

- 왼쪽 사이드바 또는 하단 패널에 Claude Code 아이콘이 나타납니다.
- `Ctrl+Shift+P` → "Claude Code" 검색 시 관련 명령어가 표시됩니다.
- 또는 `Ctrl+L` 단축키로 Claude Code 패널을 열 수 있습니다.

---

3. Claude Code 첫 실행 및 로그인

1. `Ctrl+L`을 눌러 Claude Code 패널을 엽니다.
2. 처음 실행 시 **로그인** 화면이 나타납니다.
3. 인증 방법을 선택합니다:
    - **Anthropic API 키**: API 키를 직접 입력
    - **Claude.ai 계정**: 브라우저를 통해 OAuth 로그인 (Pro/Max 구독 필요)
4. 로그인하면 채팅창이 활성화됩니다.

기본 설정 확인

Claude Code 패널에서 `/help`를 입력하면 사용 가능한 명령어 목록을 볼 수 있습니다.

---

4. 템플릿 구조 이해하기

이 템플릿은 Claude와 함께 체계적인 소프트웨어 개발을 진행하기 위한 프레임워크입니다.

폴더 구조

```
프로젝트 루트/
├── SETUP.md                        # 초기 설정 체크리스트 (완료 후 삭제)
├── docs/                           # 문서
└── .claude/                        # Claude Code 프레임워크 (핵심)
    ├── CLAUDE.md                   # 프로젝트 규칙 및 설정 (마스터 파일)
    ├── MEMORY.md                   # 메모리 인덱스
    ├── agents/                     # 전문 에이전트 (29개)
    │   ├── workflow/               # 워크플로우 에이전트 (4개)
    │   ├── panel/                  # 패널 에이전트 - 핵심 자문단 (7개)
    │   ├── product/                # 제품 전략 에이전트 (4개)
    │   ├── craft/                  # 실행 품질 에이전트 (4개)
    │   └── domains/                # 도메인 전문가 에이전트 (8개)
    ├── memory/                     # 프로젝트 메모리
    │   ├── architecture.md         # 시스템 아키텍처
    │   ├── decisions.md            # 의사결정 로그
    │   ├── feedback.md             # 워크플로우 선호도
    │   ├── lessons-learned.md      # 교훈 기록
    │   ├── deferred.md             # 보류된 항목
    │   └── spec-tracking.md        # 스펙 버전 추적
    ├── prompts/                    # 페이즈 구현 프롬프트
```

```
    │   └── prompt-template.md        # 페이즈 템플릿
    └── skills/                        # 슬래시 명령어 (5개)
```

핵심 파일 설명

파일	역할
`CLAUDE.md`	Claude의 행동 규칙, 아키텍처 원칙, 코딩 규칙 등을 정의하는 마스터 설정 파일
`MEMORY.md`	모든 메모리 파일의 인덱스. 세션 시작 시 Claude가 가장 먼저 읽는 파일
`SETUP.md`	프로젝트 초기 설정 체크리스트. 설정 완료 후 삭제

---

5. 프로젝트 초기 설정

템플릿을 새 프로젝트에 적용하려면 `SETUP.md`의 체크리스트대로 진행합니다. Claude와 대화하면서 함께 진행하는 것을 권장합니다.

단계 1: 프로젝트 기본 정보 설정

`.claude/CLAUDE.md`를 열고 아래와 같이 수정합니다:

```
# 변경 전
**Project:** PROJECT_NAME | **Stack:** <!-- FILL -->

# 변경 후 (예시)
**Project:** MyWebApp | **Stack:** React + Node.js + PostgreSQL
```

단계 2: 스펙 구조 생성

프로젝트 루트에 `Documents/Spec/` 폴더를 만들고 필수 파일을 생성합니다:

```
Documents/Spec/
├── Spec.md              # 마스터 인덱스 (v1.0부터 시작)
├── 00_Changelog.md      # 변경 이력
└── 00_IssueLog.md       # 이슈 로그
```

> 스펙은 프로젝트의 "설계 문서"입니다. Claude가 이 문서를 기준으로 코드를 작성하고 리뷰합니다.

단계 3: 아키텍처 및 테스트 설정

`CLAUDE.md`에서 `<!-- FILL -->`로 표시된 빈칸을 모두 작성합니다:

- **Communication**: 시스템 간 통신 방식 (예: REST API, event bus, Godot signals)
- **Performance**: 성능 규칙 (예: N+1 쿼리 금지, 캐시 사용, lazy loading)
- **Test command**: 테스트 실행 명령어 (예: `npm test`, `pytest`)
- **Strategy**: 테스트 전략 (예: unit + integration)

단계 4: 코드 리뷰 체크리스트 설정

`.claude/agents/workflow/code-review.md`를 열고 프로젝트에 맞게 코딩 규칙과 읽기 전용 디렉토리를 설정합니다.

단계 5: 메모리 검증

`MEMORY.md`의 링크가 정상적으로 연결되는지, `feedback.md`의 설정이 프로젝트에 적합한지 확인합니다.

단계 6: 첫 디자인 세션

설정이 끝나면 Claude와 프로젝트의 핵심 기능에 대해 대화를 시작합니다.

---

6. 핵심 워크플로우

이 템플릿의 개발 흐름은 **페이즈(Phase) 기반**입니다.

전체 흐름도

```
디자인 대화 → /design (선택) → /new-phase → 페이즈 실행 → /review → 메모리 업데이트
     ↑                                                                  ↓
     └──────────────────── 다음 페이즈 ──────────────────────────────────┘
```

상세 흐름

1단계: 디자인 대화

Claude와 자유롭게 대화하며 구현할 기능을 설계합니다.

```
사용자: 사용자 인증 시스템을 만들고 싶어. JWT 기반으로 하고,
        이메일/비밀번호 로그인과 소셜 로그인을 지원하려고 해.
```

2단계: 디자인 리뷰 (선택)

복잡한 설계 결정이 필요할 때 디자인 패널 리뷰를 실행합니다.

```
/design 인증 시스템 아키텍처
```

이 명령을 실행하면 아키텍트, 비평가, 단순화 전문가 등 3~5명의 전문가 에이전트가 각자의 관점에서 의견을 제시합니다.

3단계: 페이즈 생성

대화에서 결정된 사항을 페이즈 프롬프트로 추출합니다.

```
/new-phase 1 auth
```

이 명령은 다음을 자동으로 수행합니다:

1. 대화에서 결정 사항을 추출 및 분류
2. 스펙에 새 결정 사항 반영
3. 디자인 리뷰 실행 (선택)
4. 기존 코드베이스 스캔
5. 단계별 구현 프롬프트 생성

4단계: 페이즈 실행

생성된 프롬프트를 승인하면 Claude가 순차적으로 모든 단계를 실행합니다.

- 각 단계마다 사용자 승인을 기다리지 않고 연속 진행합니다.
- 체크포인트 생성 → 코드 작성 → 테스트 → 코드 리뷰 순으로 진행됩니다.

5단계: 코드 리뷰

```
/review
```

코드 리뷰 에이전트가 변경된 코드의 정확성, 규칙 준수 여부, 스펙과의 일치 여부를 검사합니다.

6단계: 메모리 업데이트

매 페이즈 완료 후 반드시 다음 파일을 업데이트해야 합니다:

- `memory/logs/phaseN-log.md` — 페이즈 완료 로그
- `MEMORY.md` — 현재 상태 갱신
- 필요 시: `architecture.md`, `lessons-learned.md`, `spec-tracking.md`

---

7. 에이전트 시스템

이 템플릿에는 **29개의 전문 에이전트**가 포함되어 있으며, 5개 카테고리로 나뉩니다.

워크플로우 에이전트 (workflow/)

모든 페이즈에서 사용되는 핵심 에이전트입니다.

에이전트	역할
`codebase-scan.md`	스펙 변경과 관련된 기존 코드를 탐색
`phase-planner.md`	스펙 변경 + 코드 스캔 결과를 종합하여 페이즈 프롬프트 생성
`code-review.md`	코드 정확성, 규칙 준수, 스펙 일치 여부 검사

에이전트	역할
`design-review.md`	다중 관점 설계 리뷰 (전문가 패널 소집)

패널 에이전트 (panel/) — 핵심 자문단

모든 디자인 리뷰에 참여하는 7명의 핵심 조언자입니다.

에이전트	핵심 질문
`architect.md`	"이 아키텍처가 프로젝트를 충분히 뒷받침할 수 있는가?"
`skeptic.md`	"아직 검증되지 않은 가정은 무엇인가?"
`simplifier.md`	"문제를 해결할 수 있는 가장 단순한 방법은 무엇인가?"
`user-advocate.md`	"사용자가 실제로 원하는 것을 얻을 수 있는가?"
`operator.md`	"안정적으로 배포, 모니터링, 디버깅할 수 있는가?"
`economist.md`	"한정된 자원을 가장 효율적으로 활용하고 있는가?"
`historian.md`	"같은 문제를 해결한 사람들은 어떤 교훈을 얻었는가?"

제품 에이전트 (product/) — 방향 설정 시 참여

에이전트	역할
`product-thinker.md`	문제 정의, 사용자 가치, 포지셔닝
`growth-mind.md`	채택, 유지, 바이럴리티
`storyteller.md`	메시징, 포지셔닝, 설명
`competitor-scout.md`	경쟁 환경 분석, 차별화

크래프트 에이전트 (craft/) — 실행 품질 검토 시 참여

에이전트	역할
`dx-advocate.md`	개발자 경험 (온보딩, 문서, 에러 메시지)
`guardian.md`	보안, 프라이버시, 위협 모델링
`perfectionist.md`	엣지 케이스, 에러 상태, 완성도
`scaler.md`	성능, 병목, 10x/100x/1000x 규모

도메인 에이전트 (domains/) — 하나만 활성화

프로젝트 유형에 맞는 에이전트를 하나 선택하여 맥락을 설정합니다.

에이전트	도메인
`web.md`	웹 (SPA, SSR, 브라우저)

에이전트	도메인
`mobile.md`	모바일 (iOS/Android)
`cli.md`	터미널/CLI
`api.md`	API 설계
`desktop.md`	데스크톱 앱
`library.md`	라이브러리/SDK
`pipeline.md`	CI/CD, 데이터 파이프라인
`platform.md`	인프라, 플랫폼

---

8. 스킬 (슬래시 명령어)

Claude Code 채팅창에서 슬래시(`/`)로 시작하는 명령어를 사용할 수 있습니다.

사용 가능한 스킬

명령어	설명	사용 시점
`/new-phase [N] [이름]`	디자인 대화를 페이즈 프롬프트로 변환	설계 대화 후 구현 시작 전
`/review [파일]`	변경된 코드를 리뷰	코드 작성 후
`/design [주제]`	디자인 패널 리뷰 소집	복잡한 설계 결정 시
`/update [범위]`	스펙과 코드 간 불일치를 동기화	코드 변경 후 스펙 갱신
`/audit [포커스]`	프로젝트 전체 건강 검진	정기적으로 또는 문제 감지 시

스킬 사용 예시

```
# 첫 번째 페이즈 생성
/new-phase 1 setup

# 인증 관련 코드 리뷰
/review src/auth/

# 데이터베이스 설계 리뷰
/design 데이터베이스 스키마

# 스펙 동기화
/update auth refactor

# 전체 프로젝트 감사
/audit
```

---

9. 메모리 시스템

메모리 시스템은 Claude가 여러 세션에 걸쳐 프로젝트 맥락을 유지하기 위한 핵심 기능입니다.

메모리 파일 구조

모든 메모리 파일은 `.claude/memory/`에 저장됩니다 (전역 `~/.claude/`가 아님).

파일	용도
`architecture.md`	시스템, 서비스, 데이터 구조, API, 패턴 기록
`decisions.md`	아키텍처 결정 로그 (날짜, 결정, 근거, 대안, 상태)
`feedback.md`	워크플로우 선호도 및 행동 지침
`lessons-learned.md`	작업 중 발견한 교훈 (3회 이상 반복 시 CLAUDE.md로 승격)
`deferred.md`	보류된 항목 (`[DECISION]` 또는 `[FEATURE]` 태그)
`spec-tracking.md`	스펙 버전 리뷰 기록
`logs/phaseN-log.md`	각 페이즈 완료 로그

메모리 흐름

```
세션 시작 → MEMORY.md 읽기 → 관련 메모리 파일 확인
                                          ↓
페이즈 완료 → 로그 작성 → MEMORY.md 갱신 → 관련 메모리 업데이트
```

중요 규칙

- **세션 시작 시**: Claude는 항상 `MEMORY.md`를 먼저 읽습니다.
- **페이즈 완료 후**: 메모리 업데이트는 필수입니다. 생략은 워크플로우 위반입니다.
- **교훈**: 같은 교훈이 3번 이상 반복되면 `CLAUDE.md`에 정식 규칙으로 승격됩니다.
- **보류 항목**: `/new-phase` 실행 시마다 재평가됩니다.

---

10. 실전 예시

새 프로젝트를 시작하는 전체 과정을 예시로 살펴봅니다.

예시: React 웹앱 프로젝트 시작

1. 템플릿을 프로젝트 폴더에 복사

```
# 새 프로젝트 폴더 생성
mkdir my-web-app
cd my-web-app
```

```
# 템플릿의 .claude 폴더와 SETUP.md 복사
cp -r /path/to/template/.claude .
cp /path/to/template/SETUP.md .

# Git 초기화
git init
```

2. VS Code에서 프로젝트 열기

```
code .
```

3. Claude Code 패널 열기

`Ctrl+L`을 눌러 Claude Code 채팅을 엽니다.

4. 초기 설정 요청

```
SETUP.md를 보고 프로젝트 초기 설정을 도와줘.
프로젝트 이름은 "MyWebApp"이고 스택은 React + Node.js + PostgreSQL이야.
```

Claude가 `CLAUDE.md`의 빈칸을 채우고, 스펙 파일을 생성하며, 메모리를 초기화합니다.

5. 디자인 대화

```
사용자: 이 앱은 할 일 관리 앱이야.
       사용자는 할 일을 생성, 수정, 삭제, 완료 표시할 수 있어.
       카테고리별로 분류할 수 있고, 마감일을 설정할 수 있어.
```

6. 디자인 리뷰 (선택)

```
/design 할 일 관리 데이터 모델 및 API
```

7. 첫 페이즈 생성

```
/new-phase 1 foundation
```

Claude가 자동으로:

- 대화에서 결정 사항 추출

- 스펙 파일 업데이트
- 코드베이스 스캔
- 구현 프롬프트 생성

8. 페이즈 실행 승인

생성된 프롬프트를 확인하고 승인하면, Claude가 모든 단계를 순차적으로 실행합니다.

9. 리뷰 및 다음 페이즈

```
/review
```

리뷰를 통과하면 다음 기능 설계로 넘어갑니다.

부록: 자주 묻는 질문

Q: Claude가 CLAUDE.md를 읽지 않는 것 같아요.

A: `.claude/CLAUDE.md` 파일이 프로젝트 루트의 `.claude/` 폴더 안에 있는지 확인하세요. Claude Code는 이 경로를 자동으로 인식합니다.

Q: 에이전트를 직접 호출할 수 있나요?

A: 에이전트는 스킬(`/design`, `/review` 등)을 통해 자동으로 호출됩니다. 사용자가 직접 호출할 필요는 없습니다.

Q: 사용하지 않는 에이전트를 삭제해도 되나요?

A: 네. 예를 들어 UI가 없는 프로젝트라면 `agents/design/`을, 마케팅이 필요 없다면 관련 에이전트를 삭제해도 됩니다.

Q: 페이즈 중간에 문제가 생기면?

A: Claude가 자동으로 원인을 조사하고 수정을 시도합니다. 자체적으로 해결할 수 없는 경우에만 사용자에게 보고합니다.

Q: 메모리를 수동으로 수정해도 되나요?

A: 네. `.claude/memory/` 안의 파일은 모두 일반 마크다운 파일이므로 직접 편집해도 됩니다.

부록: 유용한 단축키

단축키	기능
`Ctrl+L`	Claude Code 패널 열기/닫기
`Ctrl+Shift+P` → "Claude"	Claude Code 명령어 목록

단축키	기능
`Escape`	Claude 응답 중단
`/help`	도움말 보기
`/clear`	대화 초기화

이 매뉴얼은 VS Code용 Claude Code 확장 프로그램의 사용 가이드입니다. 최신 정보는 Claude Code 공식 문서를 참고하세요.